

PA-Q1-Part I

April 15, 2026

1 *PA - Part I: Basic Vision Models [Experiments with MNIST]* (55pt)

Keywords: Multiclass Image Classification, Neural Networks, PyTorch

MNIST * The [MNIST](#) database (Modified National Institute of Standards and Technology database) is a large database of handwritten digits that is commonly used for training various image processing systems. * The MNIST database contains 70,000 labeled images. Each datapoint is a 28×28 pixels grayscale image. * However to speed up computations, we will use a much smaller dataset with size 8×8 images. These images are loaded from `sklearn.datasets`.

Agenda: * The PA is split into three parts, the first part dealing with miniature models which we will build from scratch, the second part dealing with modern architectures and the bonus third part dealing with training vision models robust to adversarial attacks. * In this part, we will be performing multiclass classification on the simplified MNIST dataset from scratch. * We will be applying and analysing different loss functions, optimization techniques and learning methods.

- We will be using PyTorch to do most of the heavylifting for modeling and training.

Note: * Hardware acceleration (GPU) is recommended but not required for this part. * A note on working with GPU: * Take care that whenever declaring new tensors, set `device=device` in parameters. * You can also move a declared torch tensor/model to device using `.to(device)`. * To move a torch model/tensor to cpu, use `.to('cpu')` * Keep in mind that all the tensors/model involved in a computation have to be on the same device (CPU/GPU). * Run all the cells in order. * Only **add your code** to cells marked with “TODO:” or with “...” * You should not have to change variable names where provided, but you are free to if required for your implementation.

1.0.1 *Setup: Imports and Utils*

```
[1]: # imports
import torch
import numpy as np
import math
from tqdm.notebook import tqdm
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

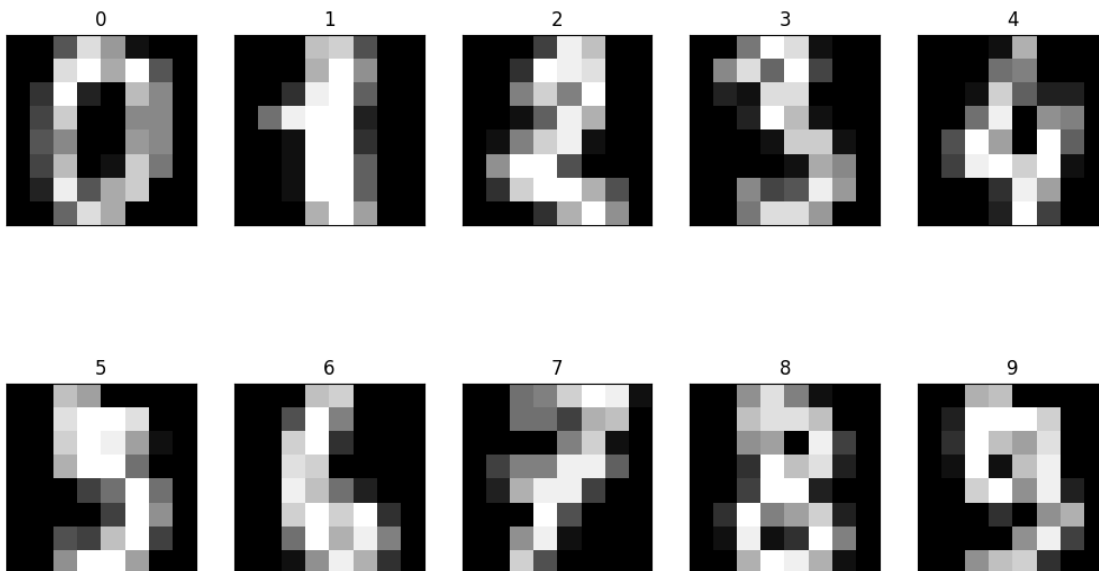
```
# loading the dataset directly from the scikit-learn library
dataset = load_digits()
X = dataset.data
y = dataset.target
print('Number of images:', X.shape[0])
print('Number of features per image:', X.shape[1])
```

Number of images: 1797

Number of features per image: 64

```
[2]: # utility function to plot gallery of images
def plot_gallery(images, titles, height, width, n_row=2, n_col=4):
    plt.figure(figsize=(2* n_col, 3 * n_row))
    plt.subplots_adjust(bottom=0, left=0.01, right=0.99, top=0.90, hspace=0.35)
    for i in range(n_row * n_col):
        plt.subplot(n_row, n_col, i + 1)
        plt.imshow(images[i].reshape((height, width)), cmap=plt.cm.gray)
        plt.title(titles[i], size=12)
        plt.xticks(())
        plt.yticks(())

# visualize some of the images of the MNIST dataset
plot_gallery(X, y, 8, 8, 2, 5)
```



```
[3]: # Let us split the dataset into training and test sets in a stratified manner.
# Note that we are not creating evaluation dataset as we will not be tuning
→hyper-parameters
```

```
# The split ratio is 4:1
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪random_state=42, stratify=y)
print('Shape of train dataset:', X_train.shape)
print('Shape of evaluation dataset:', X_test.shape)
```

Shape of train dataset: (1437, 64)
 Shape of evaluation dataset: (360, 64)

```
[4]: # define some constants - useful for later
num_classes = len(np.unique(y)) # number of target classes = 10 --
    ↪(0,1,2,3,4,5,6,7,8,9)
num_features = X.shape[1]      # number of features = 64
max_epochs = 100000           # max number of epochs for training
lr = 1e-2                     # learning rate
tolerance = 1e-6              # tolerance for early stopping during training
```

```
[5]: # Hardware Acceleration: to set device if using GPU.
# You can change runtime in colab by naviagting to (Runtime->Change runtime
    ↪type), and selecting GPU in hardware accelarator.
# NOTE that you can run this homework without GPU.
device = 'cuda' if torch.cuda.is_available() else 'cpu'
device
```

```
[5]: 'cuda'
```

1.0.2 (a) Utils and pre-processing (5pt)

- Scale the image data between 0 and 1
- One-hot encode the target data
- Convert the data to tensors and move them to the required device

```
[ ]: # 1. Scale the features between 0 and 1
# To scale, you can directly use the MinMaxScaler from sklearn.
#####

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# output variable names - X_train, X_test
#####
```

```
[8]: # 2. One-Hot encode the target labels
# To one-hot encode, you can use the OneHotEncoder from sklearn
#####

from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(sparse_output=False)
y_train_ohe = ohe.fit_transform(y_train.reshape(-1, 1))
y_test_ohe = ohe.transform(y_test.reshape(-1, 1))

# output variable names - y_train_ohe, y_test_ohe
#####
print('Shape of y_train_ohe:', y_train_ohe.shape)
print('Shape of y_test_ohe:', y_test_ohe.shape)

# move X and y to the defined device and convert them to torch.float32
X_train_torch = torch.tensor(X_train, dtype=torch.float32, device=device)
X_test_torch = torch.tensor(X_test, dtype=torch.float32, device=device)
y_train_ohe_torch = torch.tensor(y_train_ohe, dtype=torch.float32,
    ↪device=device)
y_test_ohe_torch = torch.tensor(y_test_ohe, dtype=torch.float32, device=device)

# output variable names - X_train_torch, X_test_torch, y_train_ohe_torch,
    ↪y_test_ohe_torch
```

Shape of y_train_ohe: (1437, 10)

Shape of y_test_ohe: (360, 10)

1.0.3 (b) Implement Multi-Class Logistic Regression from scratch (15pt)

In this problem, we will apply multiclass logistic regression from scratch trained using gradient descent (GD) with Mean Squared Error (MSE) loss as the objective.

We will be using a linear model

$$y^{(i)} = W\mathbf{x}^{(i)},$$

with the following notations:

- Weight matrix:

$$W_{p \times n} = \begin{bmatrix} \leftarrow & \mathbf{w}_1^\top & \rightarrow \\ \leftarrow & \mathbf{w}_2^\top & \rightarrow \\ & \vdots & \\ \leftarrow & \mathbf{w}_p^\top & \rightarrow \end{bmatrix},$$

where (p) is the number of target classes.

- Data points:

$$\mathbf{x}^{(i)} \in \mathbb{R}^n, \quad y^{(i)} \in \mathbb{R},$$

and

$$X = \begin{bmatrix} \uparrow & \uparrow & \cdots & \uparrow \\ \mathbf{x}^{(1)} & \mathbf{x}^{(2)} & \cdots & \mathbf{x}^{(m)} \\ \downarrow & \downarrow & \cdots & \downarrow \end{bmatrix}, \quad Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix},$$

where (m) is the number of datapoints.

Note: Here we need to define the model prediction. The input matrix is $X_{n \times m}$ where m is the number of examples, and n is the number of features. The linear predictions can be given by: $Y = WX + b$ where W is a $p \times n$ weight matrix and b is a p size bias vector. p is the number of target classes.

#1. Define a function `linear_model`

- This function takes as input a weight matrix (W), bias vector (b), and input data matrix of size $m \times n$ (XT).
- This function should return the predictions $\hat{y}$, which is an $m \times p$ matrix. For each datapoint row which is a $1 \times p$ vector, the prediction can be seen as the scores that the model gives each target class for that datapoint

```
[9]: #####
def linear_model(W, b, XT):
    y_hat = torch.mm(XT, W) + b
    return y_hat

#####
```

Note: The loss function that we would be using is the Mean Square Error (L2) Loss:

$MSE = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$, where m is the number of examples, $\hat{y}^{(i)}$ is the predicted value of $x^{(i)}$ and $y^{(i)}$ is the ground truth of $x^{(i)}$.

#2. Define a function `mse_loss`

- This function takes as input prediction (y_{pred}) and ground-truth label (y), and returns the MSE loss.

```
[10]: #####
def mse_loss(y_pred, y):
    loss = torch.mean((y_pred - y) ** 2)
    return loss

#####
```

#3. Define a function: initializeWeightsAndBiases In this part, we will do some setup required for training (such as initializing weights and biases) and move everything to torch tensors.

- This function returns tuple (W, b), where W is a randomly generated torch tensor of size num_classes x num_features, and b is a randomly generated torch vector of size num_classes.
- For both the tensors, set requires_grad=True in parameters.

```
[11]: #####
def initializeWeightsAndBiases(num_classes, num_features):
    W = torch.rand(num_features, num_classes, device=device, dtype=torch.
↳float32, requires_grad=True)
    b = torch.rand(num_classes, device=device, dtype=torch.float32,
↳requires_grad=True)
    return W, b
#####
```

#4 Training code

- Given below is a function: train_linear_regression_model that takes as inputs as max number of epochs (max_epochs), weights (W), biases (b), training data (X_train, y_train), learning rate (lr), tolerance for stopping (tolerance).
- This function returns a tuple (W,b,losses) where W,b are the trained weights and biases respectively, and losses is a list of tuples of loss logged every 100th epoch.
- You can go through [this](#) article for reference.

```
[12]: # Define a function train_linear_regression_model
def train_linear_regression_model(max_epochs, W, b, X_train, y_train, lr,
↳tolerance):
    losses = []
    prev_loss = float('inf')

    for epoch in tqdm(range(max_epochs)):

        #####
        # 7. do prediction
        y_pred = linear_model(W, b, X_train)

        # 8. get the loss
        loss = mse_loss(y_pred, y_train)

        # 9. backpropagate loss
        loss.backward()

        # 10. update the weights and biases
        with torch.no_grad():
            W -= lr * W.grad
            b -= lr * b.grad
```

```

# 11. set the gradients to zero
W.grad.zero_()
b.grad.zero_()

#####

# log the loss every 100th epoch and print every 5000th epoch:
if epoch%100==0:
    losses.append((epoch, loss.item()))
if epoch%5000==0:
    print('Epoch: {}, Loss: {}'.format(epoch, loss.item()))

# break if decrease in loss is less than threshold
if prev_loss - loss.item() < tolerance:
    print(f'Early stopping at epoch {epoch}')
    break
prev_loss = loss.item()

# return updated weights, biases, and logged losses
return W, b, losses

```

#5. Initialize parameters and train your own model

- Initialize weights and biases using the `initializeWeightsAndBiases` function that you defined earlier
- Train your model using function `train_linear_regression_model` defined above.
- Use full batch (set `batch_size=len(X_train)` for training (Gradient Descent).
- Also plot the graph of loss vs number of epochs (Recall that values for learning rate (`lr`) and tolerance (`tolerance`) are already defined above).

```
[14]: W, b = initializeWeightsAndBiases(num_classes, num_features)
```

```
[16]: #####
W, b = initializeWeightsAndBiases(num_classes, num_features)
W, b, losses = train_linear_regression_model(max_epochs, W, b, X_train_torch,
↪ y_train_ohe_torch, lr, tolerance)

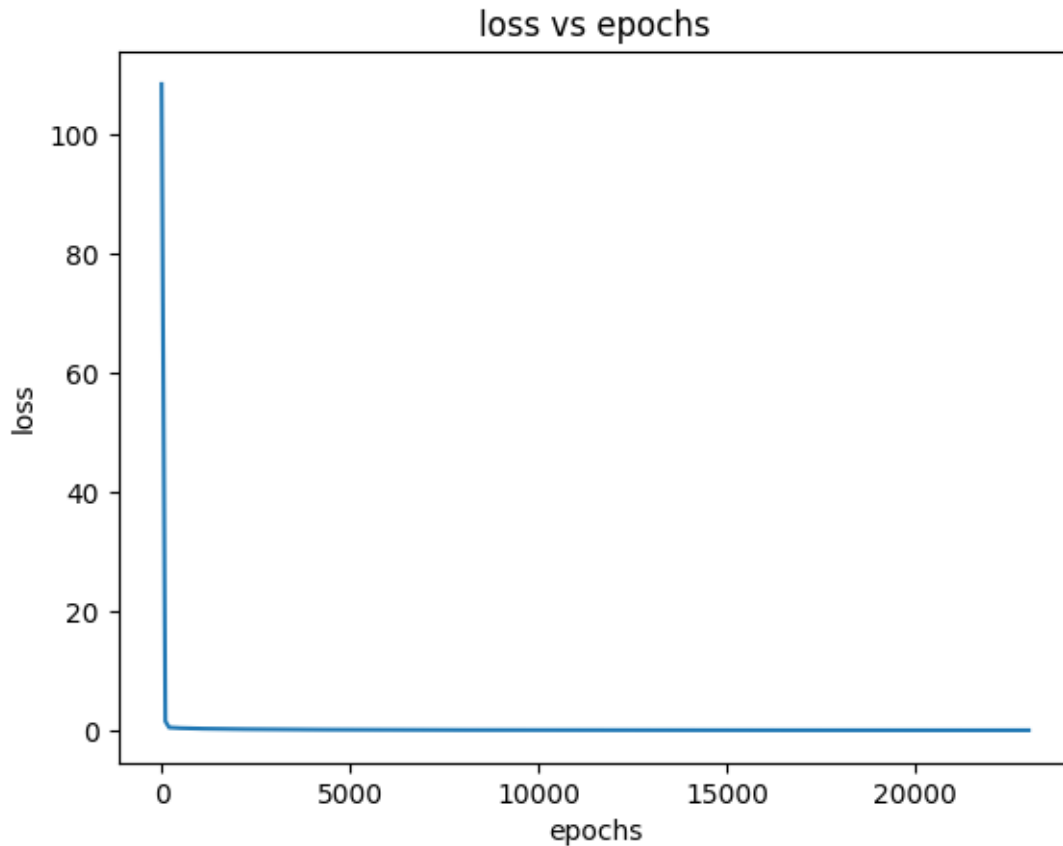
import matplotlib.pyplot as plt
plt.plot([x[0] for x in losses], [x[1] for x in losses])
plt.xlabel('epochs')
plt.ylabel('loss')
plt.title('loss vs epochs')
plt.show()

#####

```

```
0%|          | 0/100000 [00:00<?, ?it/s]
```

Epoch: 0, Loss: 108.47657775878906
Epoch: 5000, Loss: 0.13736692070960999
Epoch: 10000, Loss: 0.08372224122285843
Epoch: 15000, Loss: 0.06497304141521454
Epoch: 20000, Loss: 0.05588589236140251
Early stopping at epoch 23048



```
[18]: # print accuracies of model
predictions_train = linear_model(W, b, X_train_torch)
predictions_test = linear_model(W, b, X_test_torch)
y_train_pred = torch.argmax(predictions_train, dim=1).cpu().numpy()
y_test_pred = torch.argmax(predictions_test, dim=1).cpu().numpy()
print("Train accuracy:", accuracy_score(y_train_pred, np.asarray(y_train,
dtype=np.float32)))
print("Test accuracy:", accuracy_score(y_test_pred, np.asarray(y_test, dtype=np.
float32)))
```

Train accuracy: 0.9137091162143354
Test accuracy: 0.9305555555555556

1.0.4 (c) Use PyTorch for training (10pt)

- In the previous part, we defined the model, loss, and even the gradient update step. We also had to manually set the gradients to zero.
- In this part, we will re-implement the linear model and see how we can directly use Pytorch to do all this for us in a few simple steps.

```
[24]: # common utility function to print accuracies
def print_accuracies_torch(model, X_train_torch, X_test_torch, y_train, y_test):
    predictions_train = model(X_train_torch)
    predictions_test = model(X_test_torch)
    y_train_pred = torch.argmax(predictions_train, dim=1).cpu().numpy()
    y_test_pred = torch.argmax(predictions_test, dim=1).cpu().numpy()
    print("Train accuracy:", accuracy_score(y_train_pred, np.asarray(y_train,
    ↪dtype=np.float32)))
    print("Test accuracy:", accuracy_score(y_test_pred, np.asarray(y_test,
    ↪dtype=np.float32)))
```

#1. Define the linear model using PyTorch

- Use the inbuilt PyTorch `torch.nn.Module` to define a model class.
- Use the `torch.nn.Linear` to define the linear layer of the model

```
[20]: #####
# Define a model class using torch.nn
class Linear_Model(torch.nn.Module):
    def __init__(self):
        super(Linear_Model, self).__init__()
        # Initialize various layers of model as below
        # 1. initialize one linear layer: num_features -> num_targets
        self.linear = torch.nn.Linear(num_features, num_classes)

    def forward(self, X):
        # 2. define the feedforward algorithm of the model and return the final
        ↪output
        x = self.linear(X)
        return x

#####
```

#2. A general function for training a PyTorch model. Define a general training function: `train_torch_model`. * This function takes as input an initialized torch model (`model`), batch size (`batch_size`), initialized loss (`criterion`), max number of epochs (`max_epochs`), training data (`X_train`, `y_train`), learning rate (`lr`), tolerance for stopping (`tolerance`). * This function will return a tuple (`model`, `losses`), where `model` is the trained model, and `losses` is a list of tuples of loss logged every 100th epoch.

Note: You can reuse a lot of components from the `train_linear_regression_model` function from the previous section

You can go through [this](#) article for reference.

```
[21]: # Define a function train_torch_model
def train_torch_model(model, batch_size, criterion, max_epochs, X_train, y_train, lr, tolerance):
    losses = []
    prev_loss = float('inf')
    number_of_batches = math.ceil(len(X_train)/batch_size)

    #####
    # 3. move model to device
    model.to(device)

    # 4. define optimizer (use torch.optim.SGD (Stochastic Gradient Descent))
    # Set learning rate to lr and also set model parameters
    optimizer = torch.optim.SGD(model.parameters(), lr=lr)

    for epoch in tqdm(range(max_epochs)):
        for i in range(number_of_batches):
            X_train_batch = X_train[i*batch_size:(i+1)*batch_size].to(device)
            y_train_batch = y_train[i*batch_size:(i+1)*batch_size].to(device)

            # 5. reset gradients
            optimizer.zero_grad()

            # 6. prediction
            prediction = model(X_train_batch)

            # 7. calculate loss
            loss = criterion(prediction, y_train_batch)

            # 8. backpropagate loss
            loss.backward()

            # 9. perform a single gradient update step
            optimizer.step()
            optimizer.zero_grad()

        #####

        # log loss every 100th epoch and print every 5000th epoch:
        if epoch%100==0:
            losses.append((epoch, loss.item()))
            if epoch%5000==0:
                print('Epoch: {}, Loss: {}'.format(epoch, loss.item()))

        # break if decrease in loss is less than threshold
```

```

    if prev_loss - loss.item() < tolerance:
        print(f'Early stopping at epoch {epoch}')
        break
    prev_loss = loss.item()

# return updated model and logged losses
return model, losses

```

1.0.5 (d) Working with different model types (15pt)

- Now, we retrain the above model with `batch_size=64`
- Use Stochastic/Mini-batch Gradient Descent and keep everything else the same.
- Like before, we plot the graph between loss and number of epochs.

#1. MSE Loss and Gradient Descent (GD)

- Use `nn.MSELoss`.
- Use full batch for training (Gradient Descent).
- Also plot the graph of loss vs number of epochs.

```

[26]: #####
model = Linear_Model()
criterion = torch.nn.MSELoss()
batch_size = len(X_train_torch)
model, losses = train_torch_model(model, batch_size, criterion, max_epochs,
    ↪X_train_torch, y_train_ohe_torch, lr, tolerance)

import matplotlib.pyplot as plt
plt.plot([x[0] for x in losses], [x[1] for x in losses])
plt.xlabel('epochs')
plt.ylabel('loss')
plt.title('loss vs epochs')
plt.show()
#####

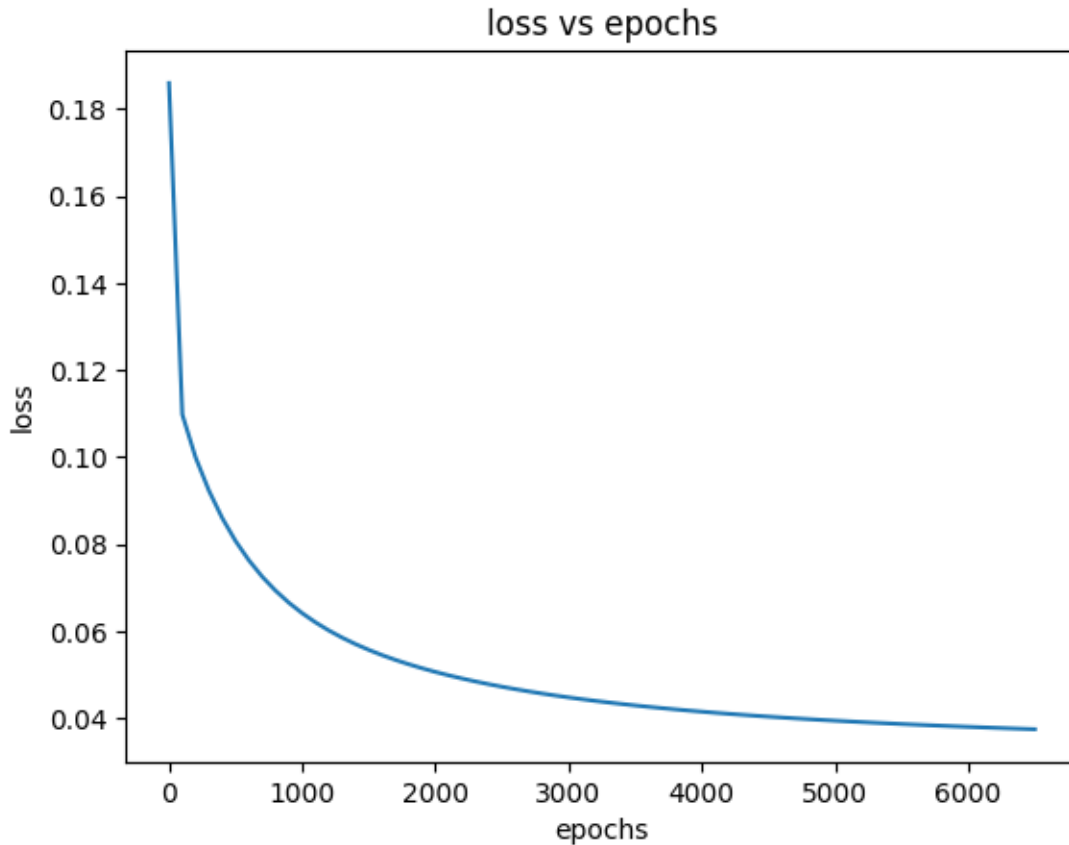
```

```
0%|          | 0/100000 [00:00<?, ?it/s]
```

Epoch: 0, Loss: 0.18589311838150024

Epoch: 5000, Loss: 0.039513543248176575

Early stopping at epoch 6554



```
[27]: # print accuracies of model
print_accuracies_torch(model, X_train_torch, X_test_torch, y_train, y_test)
```

Train accuracy: 0.9290187891440501

Test accuracy: 0.9111111111111111

#2. MSE Loss and Stochastic Gradient Descent (SGD)

- Use `nn.MSELoss`.
- Use `batch_size=64` for training
- Also plot the graph of loss vs number of epochs.

```
[28]: #####
model = Linear_Model()
criterion = torch.nn.MSELoss()
batch_size = 64
model, losses = train_torch_model(model, batch_size, criterion, max_epochs,
    ↪ X_train_torch, y_train_ohe_torch, lr, tolerance)

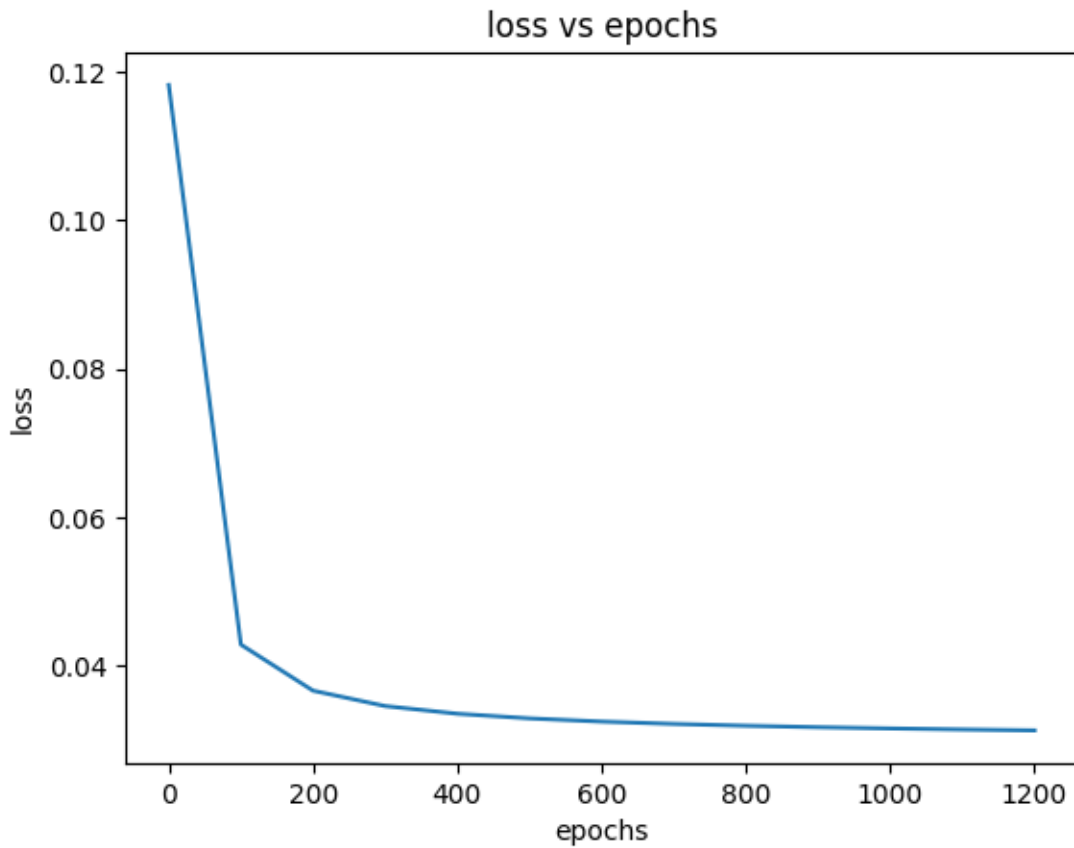
import matplotlib.pyplot as plt
```

```
plt.plot([x[0] for x in losses],[x[1] for x in losses])
plt.xlabel('epochs')
plt.ylabel('loss')
plt.title('loss vs epochs')
plt.show()
#####
```

0%| | 0/100000 [00:00<?, ?it/s]

Epoch: 0, Loss: 0.11823853105306625

Early stopping at epoch 1275



```
[29]: # print accuracies of model
print_accuracies_torch(model, X_train_torch, X_test_torch, y_train, y_test)
```

Train accuracy: 0.9457202505219207

Test accuracy: 0.9472222222222222

Cross-Entropy (CE) Loss with Linear Model

- Instead of using MSE Loss, we will use a much more natural loss function for the multi-class logistic regression task which is the Cross Entropy Loss.
- CE loss converts the model scores of each class into a probability.
- This model penalizes both choosing the wrong class as well as uncertainty (choosing the right class with low probability).
- And we will use the same linear model defined in (c).

Note: The [Cross Entropy Loss](#) for multiclass classification is the mean of the negative log likelihood of the output logits after softmax:

$$L = \frac{1}{m} \sum_{i=1}^m -y^{(i)} \log \underbrace{\frac{e^{\hat{y}^{(i)}}}{\sum_{j=1}^p e^{\hat{y}^{(j)}}}}_{\substack{\text{Softmax} \\ \text{LogSoftmax}}} ,$$

Negative Log Likelihood (NLL)
Cross Entropy (CE) Loss

where $y^{(i)}$ is the ground truth, and $\hat{y}^{(k)}$ (also called as *logits*) represent the outputs of the last linear layer of the model.

#3. CE Loss and GD

- Instead of `nn.MSELoss`, train the linear model with `nn.CrossEntropyLoss`.
- Use **full-batch**.
- Also plot the graph between loss and number of epochs.

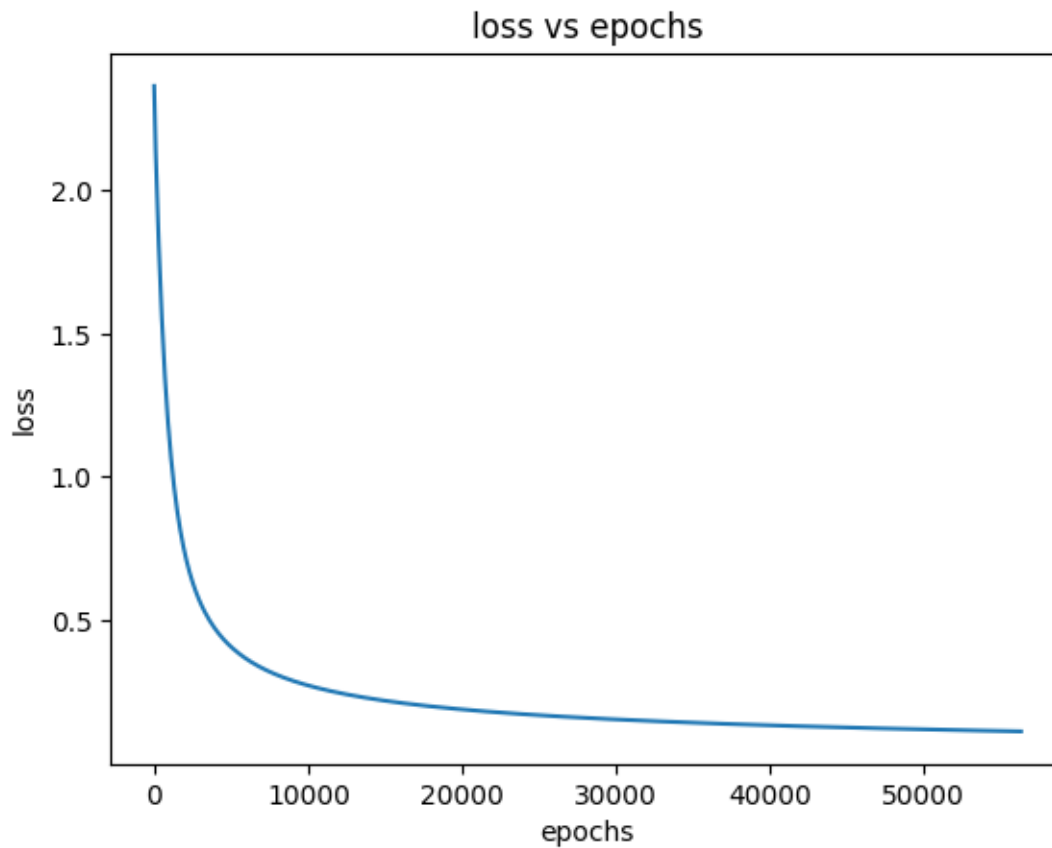
```
[30]: #####
model = Linear_Model()
criterion = torch.nn.CrossEntropyLoss()
batch_size = len(X_train_torch)
model, losses = train_torch_model(model, batch_size, criterion, max_epochs,
    ↪X_train_torch, y_train_ohe_torch, lr, tolerance)

import matplotlib.pyplot as plt
plt.plot([x[0] for x in losses], [x[1] for x in losses])
plt.xlabel('epochs')
plt.ylabel('loss')
plt.title('loss vs epochs')
plt.show()
#####
```

```
0%|          | 0/100000 [00:00<?, ?it/s]
```

```
Epoch: 0, Loss: 2.362581253051758
Epoch: 5000, Loss: 0.40804776549339294
Epoch: 10000, Loss: 0.2736990749835968
Epoch: 15000, Loss: 0.220407634973526
Epoch: 20000, Loss: 0.1901596486568451
Epoch: 25000, Loss: 0.17001718282699585
Epoch: 30000, Loss: 0.15531741082668304
Epoch: 35000, Loss: 0.14393536746501923
```

Epoch: 40000, Loss: 0.13475114107131958
Epoch: 45000, Loss: 0.12711308896541595
Epoch: 50000, Loss: 0.12061270326375961
Epoch: 55000, Loss: 0.11497961729764938
Early stopping at epoch 56363



```
[31]: # print accuracies of model
print_accuracies_torch(model, X_train_torch, X_test_torch, y_train, y_test)
```

Train accuracy: 0.9812108559498957
Test accuracy: 0.9638888888888889

#4. CE Loss and SGD

- Use a different batch size, `batch_size=64` with the new loss and repeat the previous part #1.
- Also plot the graph of loss vs epochs.

```
[32]: #####
model = Linear_Model()
criterion = torch.nn.CrossEntropyLoss()
```

```

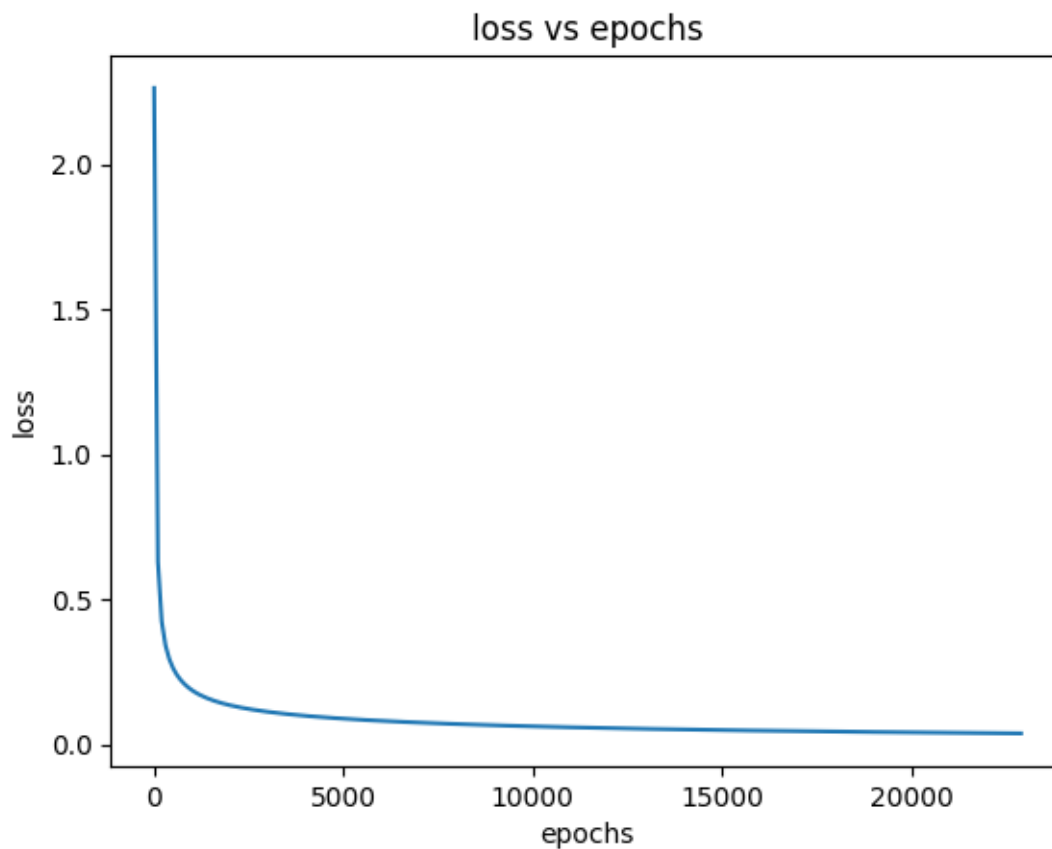
batch_size = 64
model, losses = train_torch_model(model, batch_size, criterion, max_epochs,
    ↪ X_train_torch, y_train_ohe_torch, lr, tolerance)

import matplotlib.pyplot as plt
plt.plot([x[0] for x in losses], [x[1] for x in losses])
plt.xlabel('epochs')
plt.ylabel('loss')
plt.title('loss vs epochs')
plt.show()
#####

```

0%| | 0/100000 [00:00<?, ?it/s]

Epoch: 0, Loss: 2.2636024951934814
 Epoch: 5000, Loss: 0.0880981907248497
 Epoch: 10000, Loss: 0.06155843660235405
 Epoch: 15000, Loss: 0.04867053031921387
 Epoch: 20000, Loss: 0.04074279963970184
 Early stopping at epoch 22963




```
[33]: # print accuracies of model
print_accuracies_torch(model, X_train_torch, X_test_torch, y_train, y_test)
```

Train accuracy: 0.9979123173277662

Test accuracy: 0.9694444444444444

Training a neural network model in PyTorch

- We will train a neural network in pytorch with two hidden layers of sizes 32 and 16 neurons.
- We will use non-linear ReLU activations to effectively make it a non-linear model.
- We will use this neural network model for multi-class classification with Cross Entropy Loss.

Note: The neural network model output can be represented mathematically as below: $\hat{y}_{10 \times 1}^{(i)} = W_{10 \times 16}^{(3)} \sigma(W_{16 \times 32}^{(2)} \sigma(W_{32 \times 64}^{(1)} \mathbf{x}_{64 \times 1}^{(i)} + \mathbf{b}_{32 \times 1}^{(1)}) + \mathbf{b}_{16 \times 1}^{(2)}) + \mathbf{b}_{10 \times 1}^{(3)}$, where σ represents ReLU activation, $W^{(i)}$ is the weight of the i^{th} linear layer, and $\mathbf{b}^{(i)}$ is the layer's bias. We use the subscript to denote the dimension for clarity.

#5. Define the 2 hidden-layer Neural Network (NN)

```
[34]: #####
# Define a neural network model class using torch.nn
class NN_Model(torch.nn.Module):
    def __init__(self):
        super(NN_Model, self).__init__()
        # Initialize various layers of model as instructed below
        # 1. initialize three linear layers: num_features -> 32, 32 -> 16, 16 ->
        ↪ num_targets
        self.linear1 = torch.nn.Linear(num_features, 32)
        self.linear2 = torch.nn.Linear(32, 16)
        self.linear3 = torch.nn.Linear(16, num_classes)

        # 2. initialize RELU
        self.relu = torch.nn.ReLU()

    def forward(self, X):
        # 3. define the feedforward algorithm of the model and return the final
        ↪ output
        # Apply non-linear ReLU activation between subsequent layers
        x = self.linear1(X)
        x = self.relu(x)
        x = self.linear2(x)
        x = self.relu(x)
        x = self.linear3(x)
        return x

#####
```

#6. NN with CE Loss and GD

- Use Cross Entropy Loss.
- Use full-batch and plot the graph of loss vs number of epochs.
- Note that you can re-use the training function `train_torch_model` (from part (b)).

```
[35]: #####
model = NN_Model()
criterion = torch.nn.CrossEntropyLoss()
batch_size = len(X_train_torch)
model, losses = train_torch_model(model, batch_size, criterion, max_epochs,
    ↪X_train_torch, y_train_ohe_torch, lr, tolerance)

import matplotlib.pyplot as plt
plt.plot([x[0] for x in losses], [x[1] for x in losses])
plt.xlabel('epochs')
plt.ylabel('loss')
plt.title('loss vs epochs')
plt.show()
#####
```

```
0%|          | 0/100000 [00:00<?, ?it/s]
```

Epoch: 0, Loss: 2.3167550563812256

Epoch: 5000, Loss: 0.1832629293203354

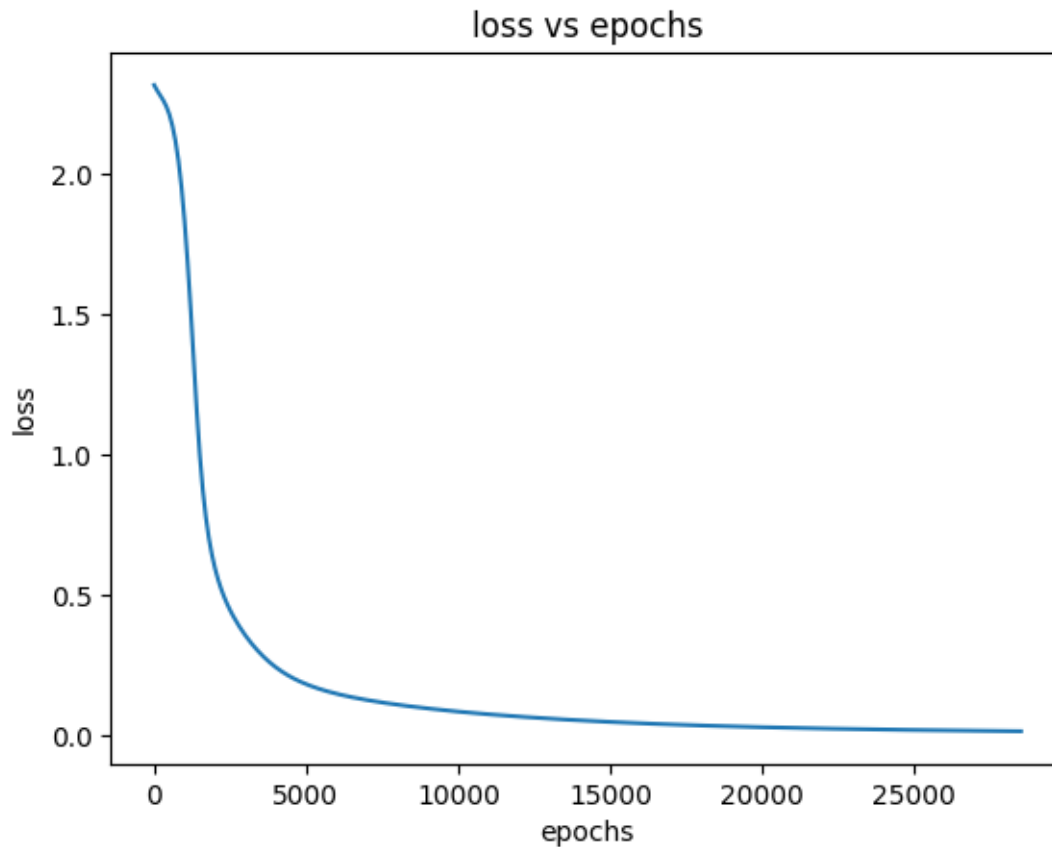
Epoch: 10000, Loss: 0.08558323234319687

Epoch: 15000, Loss: 0.048801910132169724

Epoch: 20000, Loss: 0.030072282999753952

Epoch: 25000, Loss: 0.02008567377924919

Early stopping at epoch 28593



```
[36]: # print accuracies of model
print_accuracies_torch(model, X_train_torch, X_test_torch, y_train, y_test)
```

Train accuracy: 0.9993041057759221

Test accuracy: 0.9638888888888889

#7. NN with CE Loss and SGD

- Re-train the above model with `batch_size=64`.
- Also plot the graph of loss vs epochs.

```
[37]: #####
model = NN_Model()
criterion = torch.nn.CrossEntropyLoss()
batch_size = 64
model, losses = train_torch_model(model, batch_size, criterion, max_epochs,
    ↪ X_train_torch, y_train_ohe_torch, lr, tolerance)

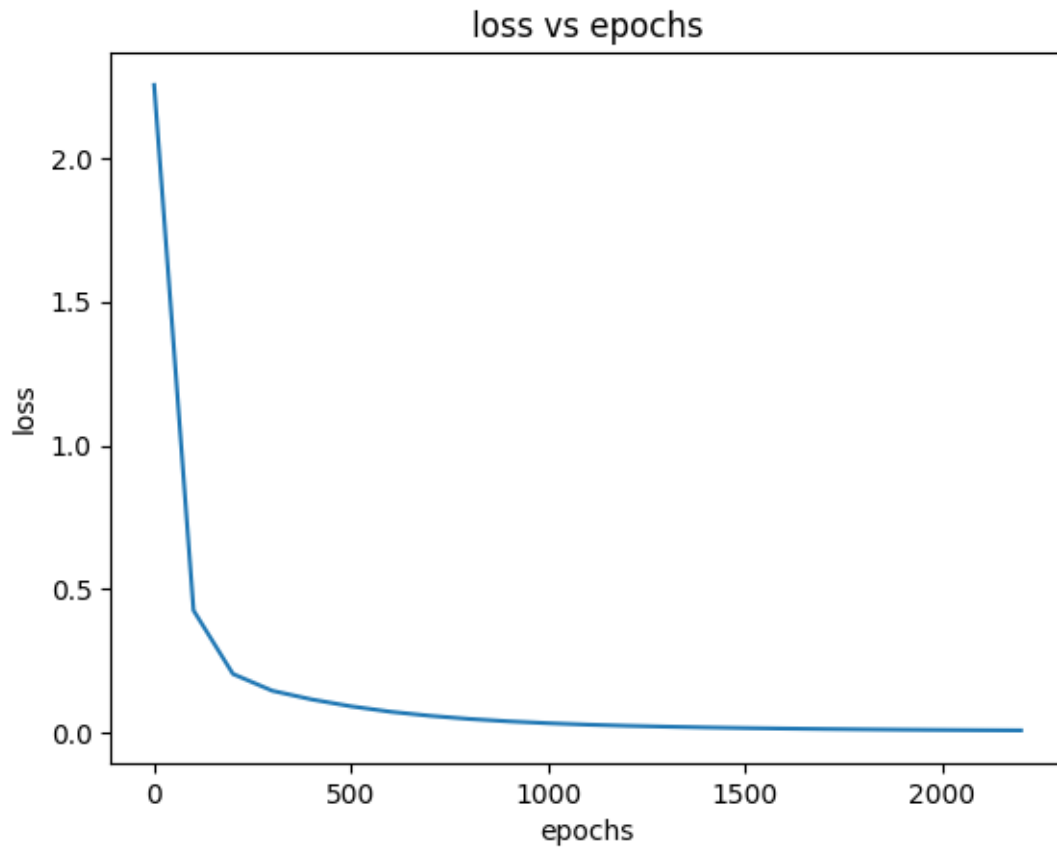
import matplotlib.pyplot as plt
plt.plot([x[0] for x in losses], [x[1] for x in losses])
```

```
plt.xlabel('epochs')
plt.ylabel('loss')
plt.title('loss vs epochs')
plt.show()
#####
```

0%| | 0/100000 [00:00<?, ?it/s]

Epoch: 0, Loss: 2.254425287246704

Early stopping at epoch 2237



```
[38]: # print accuracies of model
print_accuracies_torch(model, X_train_torch, X_test_torch, y_train, y_test)
```

Train accuracy: 1.0

Test accuracy: 0.9805555555555555

1.0.6 (e) Analyze the results (10pt)

In the above few examples, we performed several experiments with different batch size and loss functions. Now it's time to analyze our observations from the results.

Recall that we trained the following models in this tutorial:

1. Linear Model - Scratch + MSE Loss + Full Batch (GD)
2. Linear Model - PyTorch + MSE Loss + Full Batch
3. Linear Model - PyTorch + MSE Loss + Mini Batch (SGD)
4. Linear Model - PyTorch + CE Loss + Full Batch
5. Linear Model - PyTorch + CE Loss + Mini Batch
6. NN Model - PyTorch + CE Loss + Full Batch
7. NN Model - PyTorch + CE Loss + Mini Batch

```
[39]: # plotting graph of accuracy vs model
import pandas as pd
import plotly.express as px
xlabels = ['linear-scratch', 'linear-torch-full', 'linear-torch-batch',
           'linear-celoss-full', 'linear-celoss-batch', 'nn-full', 'nn-batch']
train_accuracies = [0.9137091162143354, 0.9290187891440501, 0.9457202505219207,
                    0.9812108559498957, 0.9979123173277662, 0.9993041057759221, 1.0]
test_accuracies = [0.9305555555555556, 0.9111111111111111, 0.9472222222222222,
                   0.9638888888888889, 0.9694444444444444, 0.9638888888888889, 0.9805555555555555]

tuples = [(xlabels[i], train_accuracies[i], 'train' ) for i in
           range(len(xlabels))] + [(xlabels[i], test_accuracies[i], 'test' ) for i in
           range(len(xlabels))]
df = pd.DataFrame(tuples, columns = ['model', 'accuracy', 'type'])
px.line(df, x='model', y='accuracy', color = 'type', markers=True, title =
        'accuracy vs. model')
```

#1. Analysis Effect of using full vs. batch gradient descent:

Batch GD always outperformed its full batch counterpart in terms of accuracy. Batch GD also reached a better accuracy faster than its full batch counterpart. This is expected as full batch (GD) uses the entire training dataset to compute the gradient at each step, leading to a stable loss curve but extremely slow updates. On the otherhand, batched GD (SGD) has noisier gradients as there are significantly more updates in batches per epoch.

Effect of using different loss strategy:

MSE treats the problem like linear regression, which doesn't exactly align well for a classification problem like MNIST. Consequently, we do see that all CE loss models perform better than the MSE models with higher training and testing accuracies.

Effect of using linear vs. non-linear models:

Arguably MNIST is not linearly separable however the CE loss models performed decently. Still, the NN models performed better as these models can learn complex patterns related to each digit.

Training time per epoch in different cases:

Batch GD was always slower compared to full batch, as I was using GPU. With a dataset size of 1437 and batch size of 64, we are running about 22x more computations than the full batch, which ultimately is minimal in terms of GPU usage. Consequently the overhead of the 22 computations is significant compared to the single full batch computation. This is the case for both total runtime and training time per epoch. The linear models were the fastest compared to the NN models, which is also expected due to the additional multiple linear + ReLu layer computations.